DSA Roadmap

1. Language Selection

- Pick one language (Java, C++, Python) and ensure a solid grasp of its syntax and libraries relevant to DSA.

2. Programming Foundations

- **Master the Basics**
    - Data Types
    - Variables
    - Operators
    - Conditional Statements
    - Loops
    - Functions
- **Understand Object-Oriented Programming (OOP)**
    - Class & Objects
    - Inheritance
    - Abstraction
    - Encapsulation
    - Polymorphism

3. Linear Data Structures

- Arrays
- Linked Lists (Singly and Doubly Linked Lists)
- Stack
- Queues (including Circular Queue and Deque)
- Hash Maps / Hash Tables

4. Time and Space Complexities

- Time Complexity (Big-O, Big-Theta, Big-Omega)
- Space Complexity
- Methods to Calculate Complexities (Iterative and Recursive Analysis)

5. Sorting Algorithms

- Bubble Sort
- Insertion Sort

- Selection Sort
- Merge Sort
- Quick Sort
- Heap Sort

6. Searching Algorithms

- Linear Search
- Binary Search

7. Tree Data Structures

- **Concepts**
    - Binary Trees
    - Binary Search Trees
    - AVL Trees
    - B Trees
- **Tree Traversal**
    - Pre-order Traversal
    - In-order Traversal
    - Post-order Traversal
    - Level-order Traversal
- **Algorithms**
    - Breadth-First Search (BFS)
    - Depth-First Search (DFS)

8. Graph Data Structures

- **Representation**
    - Adjacency Matrix
    - Adjacency List
- **Directed Graphs**
    - Topological Sorting (including Kahn's Algorithm)
    - Cycle Detection
- **Undirected Graphs**
    - Dijkstra's Algorithm
    - Spanning Trees
    - Minimum Spanning Tree (Prim's and Kruskal's Algorithms)

- **Graph Traversal**
    - Breadth-First Traversal
    - Depth-First Traversal

9. Complex and Advanced Data Structures

- Tries
- B/B+ Trees
- Segment Trees
- Fenwick Trees (Binary Indexed Trees)
- Tree-Based Indexing

Problem-Solving Techniques

- Recursion
- Brute Force
- Two Pointer Technique
- Greedy Algorithms
- Sliding Window Technique
- Divide and Conquer
- Dynamic Programming (1D and 2D problems)
- Backtracking

Useful Resources:

Platforms To Practice DSA:
Leetcode - https://leetcode.com/
Codechef - https://www.codechef.com/
Hackerrank - https://www.hackerrank.com/

Video Tutorial: freeCodeCamp's DSA Course

Learn and Practice DSA - Strivers A2Z DSA Course

Visualise Data Structures and Algorithms:
https://visualgo.net/en

https://www.cs.usfca.edu/~galles/visualization/Algorithms.html

System Design Roadmap

1. Fundamentals of System Design

- **Basic Computer Science Knowledge**:
    - Networking: TCP/IP, HTTP/HTTPS, DNS, Load Balancers.
    - Operating Systems: Processes, Threads, Sockets, File Systems.
    - Databases: SQL, NoSQL, ACID, BASE.
- **Scalability**:
    - Vertical vs. Horizontal Scaling
    - CAP Theorem.
- **System Design Basics**:
    - Monolithic vs. Microservices
    - APIs: REST, GraphQL, gRPC

2. Core Components

- **Databases**:
    - SQL: MySQL, PostgreSQL.
    - NoSQL: MongoDB, Cassandra, Redis.
    - Sharding, Indexing, Query Optimization
- **Caching**:
    - Client-Side, Server-Side, Distributed Caching
    - Tools: Redis, Memcached
- **Load Balancing**:
    - Algorithms: Round Robin, Least Connections
    - Tools: Nginx, HAProxy, AWS ELB
- **Message Queues**:
    - Concepts: Pub/Sub, Dead Letter Queues
    - Tools: Kafka, RabbitMQ, Amazon SQS
- **Storage Solutions**:
    - Block Storage, Object Storage (e.g., S3), CDNs

3. High-Level Design (HLD)

- **System Architecture**:

    - Monolithic vs. Microservices.
    - Event-Driven vs. Request-Response
- **System Components**:
    - Databases
    - Caching
    - Load Balancers
    - APIs
- **Component Interaction**:
    - Communication Protocols:
        - REST
        - RPC
        - gRPC
- **Scalability & Fault Tolerance**:
    - Horizontal/Vertical Scaling
    - Redundancy
- **Security**:
    - Authentication
    - Encryption (SSL/TLS)

4. Low-Level Design (LLD)

- **Data Structures**:
    - Arrays
    - Trees
    - Graphs
    - Tries
    - Segment Trees
- **Algorithms**:
    - Sorting
    - Searching
    - BFS
    - DFS
- **Class Design**:
    - OOP Principles
    - UML Diagrams.
- **Code Efficiency**:

    - Time/Space Optimization
    - Profiling

5. Advanced Concepts

- **Distributed Systems**:
    - Leader Election
    - Consensus (Paxos, Raft)
- **Scalability & High Availability**:
    - Auto-Scaling
    - Failover Mechanisms
- **Data Engineering**:
    - Batch vs. Stream Processing
    - Apache Spark
- **Observability**:
    - Metrics: Prometheus.
    - Logging: ELK Stack.
    - Tracing: Jaeger.

Real-World System Design

- **Patterns**:
    - Event-Driven
    - CQRS
    - Saga
- **Popular Problems**:
    - URL Shortener
    - File Storage
    - Notification System
    - Ride-Sharing
- **Security**:
    - OAuth
    - JWT
    - API Security

Useful Resources

1. **Visualize Distributed Systems Concepts**
    - Raft Consensus Animation.
    - System Design Visualizations

2. **Video Tutorials**
    - System Design YouTube Series by Gaurav Sen
    - freeCodeCamp's System Design Playlist
    - High Level Design tutorial Playlist
    - Low Level Design tutorial Playlist

3. **System Design Interview**
    - Read: https://drive.google.com/file/d/1ka3Vd5jk0zJVllzlHGJ-4qfUhutPsMCp/view

4. **Course**
    - Complete System Design Videos by Striver

5. **Mock Interviews**
    - Pramp - https://www.pramp.com/dev/uc-system-design
    - InterviewBit - https://www.interviewbit.com/system-design-interview-questions/